A PROJECT REPORT

ON

FOUNDATION COURSE
(TINKERING & ELEMENTS OF DESIGN)

SMART LOST & FOUND SYSTEM

Submitted in partial fulfillment of the requirement for the award of Degree of
Bachelor of Technology in Artificial Intelligence & Data Science

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Mitesh Nehra	24EARAD102
Mohit Kumar Sharma	24EARAD103
Naitik Pathak	24EARAD106
Paramveer Singh	24EARAD119
Prabhash Kumar Chaudhary	24EARAD121

Session:2026-27

Department of Artificial Intelligence & Data Science
ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIALAREA, KUKAS, JAIPUR

Department of Artificial Intelligence & Data Science

CERTIFICATE OF APPROVAL

The Project Report entitled "**SMART LOST & FOUND SYSTEM**" submitted by "**Mitesh Nehra (24EARAD102), Mohit Kumar Sharma (24EARAD103), Naitik Pathak (24EARAD106), Paramveer Singh (24EARAD119), Prabhash Kumar Chaudhary (24EARAD121)**" has been examined by us and is hereby approved for carrying out the project leading to the award of degree "**Bachelor of Technology in Artificial Intelligence & Data Science**" . By this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the pursuance of project only for the above mentioned purpose.

Mentor
Er. Ram Babu Buri
Assistant Professor

Department of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that the work embodies in this Proect entitled "**SMART LOST & FOUND SYSTEM**" submitted by "**Mitesh Nehra (24EARAD102), Mohit Kumar Sharma (24EARAD103), Naitik Pathak (24EARAD106), Paramveer Singh (24EARAD119), Prabhash Kumar Chaudhary (24EARAD121)**" in partial fulfillment of the requirement for the award of "**Bachelor of Technology in Artificial Intelligence & Data Science**" to Rajasthan Technical University, Kota(Raj.) during the academic year 2026-27 is a record of bonafide piece of work, carried out by them under our supervision and guidance in the "**Department of Artificial Intelligence & Data Science**" , **Arya College of Engineering & Information Technology, Jaipur.**

Mentor
Er. Ram Babu Buri
Assistant Professor

HOD
Dr. Ashok Kajla
Professor

Department of Artificial Intelligence & Data Science

DECLARATION

We " **Mitesh Nehra, Mohit Kumar Sharma, Naitik Pathak, Paramveer Singh and Prabhash Kumar Chaudhary**" , students of **Bachelor of Technology in Artificial Intelligence & Data Science, session 2026-27, Arya College of Engineering & Information Technology, Jaipur**, here by declare that the work presented in this Project entitled "**SMART LOST & FOUND SYSTEM**" is the outcome of our own work, is bonafide and correct to the best of our knowledge and this work has been carried out taking care of Engineering Ethics. The work presented does not infringe any patented work and has not been submitted to any other University or anywhere else for the award of any degree or any professional diploma.

Mitesh Nehra	24EARAD102
Mohit Kumar Sharma	24EARAD103
Naitik Pathak	24EARAD106
Paramveer Singh	24EARAD119
Prabhash Kumar Chaudhary	24EARAD121

Date : 10-10-2026

ACKNOWLEDGEMENT

I would like to express my special thanks of gratitude to my Project Guide and Project Coordinator who gave me the golden opportunity to do this wonderful project on the topic "**SMART LOST & FOUND SYSTEM**", which also helped me in doing a lot of research and I came to know about so many new things. I am really thankful to them. Secondly, I would also like to thank my parents and friends who helped me a lot in finalizing this project within the limited time frame. I am overwhelmed in all humbleness and gratefulness to acknowledge my depth to all those who have helped me to put these ideas, well above the level of simplicity and into something concrete. Any attempt at any level cannot be satisfactorily completed without the support and guidance of my parents and friends. I would like to thank them who helped me a lot in gathering different information, collecting data and guiding me from time to time in making this project unique.

Thanking you

ABSTRACT

Misplacing personal belongings such as student identity cards, laboratory manuals, electronic gadgets, backpacks, and keys is an enduring logistical problem across bustling university campuses. Conventional campus recovery approaches rely on physical pinboards, informal social media chat groups, and fragmented reception counters. These outdated mechanisms suffer from acute information asymmetry, zero centralized tracking, vulnerable verification processes, and substantial administrative latency.

The **Smart Lost & Found System** resolves these deficiencies by introducing a high-performance, centralized desktop application engineered with **Core Java (JDK 17/8+), Java Swing GUI, and MySQL relational persistence managed through JDBC and the Data Access Object (DAO) pattern**.

The system implements strict Role-Based Access Control (RBAC) across two synchronized interfaces:

1. **Student Self-Service Panel**: enables students to log lost items with rich metadata (name, category, location, date, contact), deposit found articles, monitor real-time approval states with custom status color badges, receive automated administrative approval notifications, and execute one-click item claims.

2. **Administrative Governance Console**: empowers campus administrators to oversee registered students, audit and process pending lost and found item queues, execute single-click approvals with instant student notification triggers, and maintain a historical audit registry of resolved claims.

Furthermore, the system features an intelligent **Multi-Attribute Smart Matching Algorithm** (MatchingService.java) that applies rule-based lexical similarity across item titles and categories to automate match discovery between lost reports and recovered inventory. Packaged as a portable Windows executable (SmartLostAndFound.exe) and runnable fat JAR (SmartLostAndFound.jar), the platform delivers fast query response, ACID-compliant transactions, parameterized defense against SQL Injection, and zero-configuration database bootstrapping.

Table of Contents

1. Objective & Scope of the Project 1
1.1 Objective 1
1.2 Scope 1
1.3 Limitations 1
2. Theoretical Background 2
2.1 Java and Swing 2
2.2 JDBC and MySQL 2
2.3 DAO Pattern 2
2.4 Role-Based Access 2
2.5 Rule-Based Matching 3
3. Definition of Problem 3
3.1 Proposed Solution 3
4. System Analysis & User Requirements 4
4.1 Existing System 4
4.2 Proposed System 4
4.3 Functional Requirements 4
4.4 Non-Functional Requirements 5
4.5 User Classes 6
5. System Planning (PERT Chart) 6
6. Methodology Adopted & Details of Hardware and Software Used 7
6.1 Methodology 7
6.2 Hardware Requirements 8
6.3 Software Requirements 8
6.4 Architecture 9
7. Detailed Life Cycle of the Project 9
7.1 Planning 9

7.2 Requirement Analysis....9
7.3 System Design....9
7.4 Implementation....10
7.5 Integration....10
7.6 Testing....10
7.7 Documentation and Delivery....10
7.8 Team Module Allocation....10
8. ERD, DFD....11
8.1 Entity Relationship Diagram (ERD)....11
8.2 Data Flow Diagram — Level 0....12
8.3 Data Flow Diagram — Level 1....12
9. Process Involved, Algorithm, Flowchart, Database Diagram....13
9.1 Main Process....13
9.2 Matching Algorithm....13
9.3 Login Flowchart....14
9.4 Report and Claim Flow....15
9.5 Database Tables....16
10. Input and Output Screen Design (Screenshots)....16
10.1 Login and Student Registration — Input Screens....17
10.2 Student Report Lost Item — Input Screen....18
10.3 Student Report Found Item — Input Screen....19
10.4 Student Notifications & Approvals — Output Screen....20
10.5 Student My Claimed Items — Output Screen....21
10.6 Admin Registered Users — Output Screen....22
10.7 Admin Register Found Items & Approval — Input/Output Screen....23
10.8 Admin Campus Desk Found Item Entry — Input Screen....24

10.9 Admin Lost Items Approval — Output/Review Screen 25
10.10 Admin Claimed Items Details — Output Screen 26
11. Printout of the Code Sheet (Coding) 27
11.1 Main Components 27
11.2 Representative JDBC Pattern 28
12. Testing 29
12.1 Types of Testing 31
13. User / Operational Manual 31
13.1 Prerequisites 31
13.2 Starting the Application 31
13.3 Student Operations 32
13.4 Administrator Operations 32
13.5 Security, Access Rights, Backup and Controls 32
14. Conclusions 34
15. Future Enhancement 35
16. References 36

1. Objective & Scope of the Project

1.1 Objective

The objective of the Smart Lost & Found System is to provide a centralized desktop application for managing lost and found belongings within a college campus. It allows students to report lost items, report items they have found, search available records, view possible matches, and follow the status of a claim. Administrators can review submitted reports and claims and update their status.

1.2 Scope

- Student registration and login with role-based access for students and administrators.
- Forms for submitting lost-item and found-item details, including item name, category, location, date, description, and contact information.
- Database-backed lists and search facilities for viewing relevant item records.
- Rule-based matching that compares item name, category, location, and description and produces a score from 0 to 100.
- Claim submission and status tracking, with administrative review and remarks where applicable.
- Persistent storage using MySQL through JDBC and a layered Java application structure.

1.3 Limitations

The application is intended for a campus environment and depends on a configured MySQL database. Matching is rule-based, not artificial intelligence trained on a dataset, and a suggested match is not proof of ownership. Final verification should be performed by the administrator and/or through claimant-provided details.

2. Theoretical Background

2.1 Java and Swing

Java is an object-oriented programming language used to implement the application's models, business logic, database access, and event handling. Java Swing provides desktop GUI components such as JFrame, JPanel, JButton, JTextField, JTable, JScrollPane, and dialog boxes.

2.2 JDBC and MySQL

JDBC (Java Database Connectivity) is the Java API used to connect to a relational database, execute parameterized SQL statements, and read query results. MySQL stores users, lost items, found items, and claims in related tables. Primary keys uniquely identify records, while foreign keys preserve relationships between records.

2.3 DAO Pattern

The Data Access Object pattern separates SQL and persistence operations from the user interface. UserDAO, LostItemDAO, FoundItemDAO, and ClaimDAO provide methods for the corresponding database entities. This separation makes the project easier to test, maintain, and explain.

2.4 Role-Based Access

The system distinguishes Student and Admin roles. Students perform user-level tasks; administrators review records and manage the approval/claim workflow. The application should enforce permissions in event handlers and data-access operations, not only by hiding buttons.

2.5 Rule-Based Matching

The MatchingService compares attributes of lost and found item records. The README specifies a weighted score: item name up to 30 points, category up to 20, location up to 25, and description up to 25. The score indicates similarity and should be treated as a lead for manual verification, not an automatic confirmation.

3. Definition of Problem

In a college campus, students may lose items such as identity cards, books, wallets, keys, electronic devices, or accessories. Information about a lost item may be shared informally through classmates, notice boards, or messaging groups. Such information can be scattered, difficult to search, and quickly overlooked.

The project addresses the absence of a single organized record of lost and found items. It aims to make reporting consistent, keep item status visible, support searching and potential-match discovery, and provide an administrative review process.

3.1 Proposed Solution

The proposed system stores reports in a central MySQL database. Students use a Java Swing interface to submit and track records. The application compares item details to identify possible matches. Administrative screens provide a controlled way to review reports and claims and update their status.

4. System Analysis & User Requirements

4.1 Existing System

The existing informal process generally relies on manual announcements, direct communication, or scattered online posts. Searching old messages and confirming ownership can be time-consuming.

4.2 Proposed System

The proposed application maintains searchable, structured records and presents different dashboards according to the user's role. Data is persisted in MySQL so that records remain available after the application is closed.

4.3 Functional Requirements

ID	Requirement
FR-01	A student can register with the required identity and contact details.
FR-02	A registered user can log in and the system opens the dashboard appropriate to the role.
FR-03	A student can submit a lost-item report.
FR-04	A student can submit a found-item report.
FR-05	A student can search or view item records available to them.

FR-06	The system can calculate and display possible lost/found matches.
FR-07	A student can submit a claim with supporting proof details and view its status.
FR-08	An administrator can review item records and claims and approve/reject or update status.
FR-09	The system stores and retrieves records through JDBC and MySQL.
FR-10	A user can log out and end the active session.

4.4 Non-Functional Requirements

Usability: forms and navigation should be understandable to college students.

Reliability: database errors should be handled with clear feedback rather than crashing the application.

Security: parameterized SQL should be used; access to administrative actions should be restricted.

Maintainability: UI, model, service, and DAO responsibilities should remain separated.

Performance: common screens and searches should respond within a reasonable time on a standard college computer.

Portability: the application should run on a compatible Windows system with a supported JDK and MySQL service.

4.5 User Classes

User	Main Permissions
Student	Register/login, report lost/found items, search records, view possible matches, submit claims, track status, logout.
Administrator	Login, review item reports and claims, approve/reject or update statuses, view records and relevant user details.

5. System Planning (PERT Chart)

The project plan is organized into 12 weeks from 6 July to 27 September 2026. The sequence below is a simplified PERT-style dependency view; estimated durations are planning estimates rather than measured execution times.

Activity	Duration	Depends on	Deliverable
A. Team formation and abstract	1 week	—	Approved project idea and team roles
B. SRS and requirements	1 week	A	Functional/non-functional requirements
C. UML and database design	1 week	B	Use-case, class, ER and flow diagrams
D. UI mockups and schema	1 week	C	Screens and database tables

E. Authentication + DAO coding	2 weeks	D	Login/register and persistence layer
F. Student and admin panels	2 weeks	D, E (integration points)	Student/admin workflows
G. Integration and testing	2 weeks	E, F	Integrated application and defect fixes
H. Report, PPT and demo	1 week	G	Documentation and presentation
I. Final review and packaging	1 week	H	Final deliverables

The detailed daily allocation is maintained in the team's 12-week project plan. Team responsibilities are divided into Database & DAO, Authentication & User Management, Student Panel, Admin Panel, and Testing/Integration/Documentation.

6. Methodology Adopted & Details of Hardware and Software Used

6.1 Methodology

An incremental development approach was followed: requirements were identified first, followed by design, module-wise implementation, integration, testing, and documentation. Each team member worked on a defined module and the modules were integrated around shared model classes and database operations.

6.2 Hardware Requirements

Component	Minimum practical requirement
Processor	Dual-core processor or better
RAM	4 GB minimum; 8 GB recommended for development
Storage	At least 1 GB free space for project, JDK, connector and database
Display	1366 × 768 or higher recommended
Network	Not required for local execution; useful for downloads/version control

6.3 Software Requirements

Software	Purpose
Windows 10/11	Target desktop environment
JDK (project README specifies JDK 17 for build)	Compile and run Java source
Java Swing	Desktop user interface
MySQL Server	Relational database
MySQL Connector/J	JDBC driver connecting Java to MySQL
Visual Studio Code / compatible Java IDE	Editing, compiling and debugging
Git and GitHub	Version control and team collaboration
draw.io or equivalent	Preparing ERD, DFD, UML and flowcharts

6.4 Architecture

Student / Administrator

↓

Java Swing UI (LoginFrame, RegisterDialog, DashboardFrame)

↓

Model Classes (User, LostItem, FoundItem, Claim)

↓

DAO Classes (UserDAO, LostItemDAO, FoundItemDAO, ClaimDAO)

↓

DatabaseConnection / JDBC

↓

MySQL: users | lost_items | found_items | claims

7. Detailed Life Cycle of the Project

7.1 Planning

The team selected the campus lost-and-found problem, assigned five modules, prepared an abstract, and defined the target users and main workflows.

7.2 Requirement Analysis

The team documented student and administrator actions, form fields, status changes, data constraints, and non-functional expectations.

7.3 System Design

Model classes, DAO responsibilities, database relationships, screen layouts, and activity/data-flow diagrams were planned before full implementation.

7.4 Implementation

The UI was developed using Swing. Model classes represent users, lost items, found items, and claims. DAO classes perform SQL operations using JDBC. MatchingService handles rule-based similarity scoring.

7.5 Integration

Authentication, student actions, admin review, matching, and database persistence were connected through shared models and DAO methods.

7.6 Testing

Individual forms and database methods were checked, followed by workflow and integration checks such as report submission, review, claim submission, and status visibility.

7.7 Documentation and Delivery

The team prepared this report, supporting diagrams, screenshots, code listings, presentation material, and setup instructions.

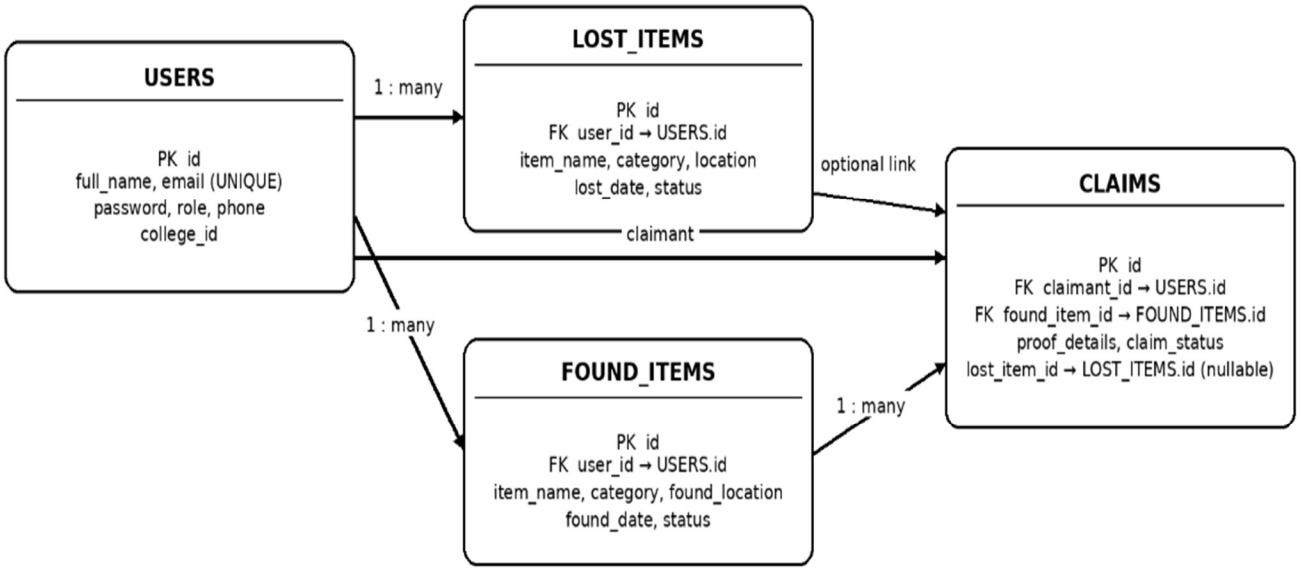
7.8 Team Module Allocation

Member	Module	Main responsibility
Mohit Kumar Sharma	Database & DAO Layer	Database schema, JDBC connection, DAO operations, ER diagram
Prabhash Kumar Chaudhary	Authentication & User Management	Login, registration, session and role handling

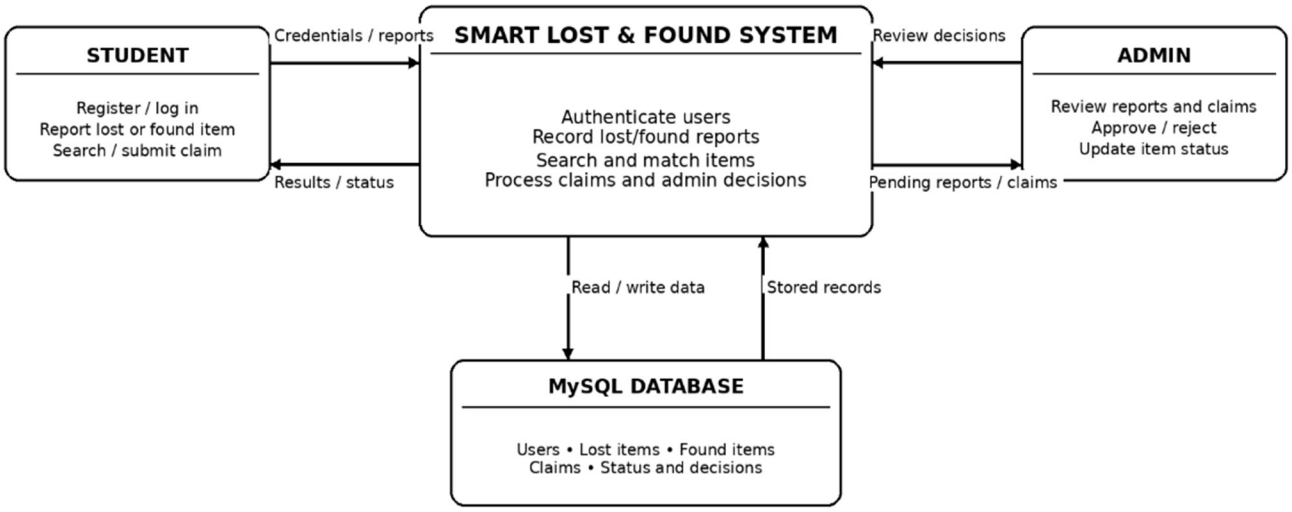
Mitesh Nehra	Student Panel (UI + Logic)	Student reporting, search/matches, claims and status views
Naitik Pathak	Admin Panel (UI + Logic)	Administrative review, status updates and record views
Paramveer Singh	Testing, Integration & Documentation	Test cases, integration coordination, report and presentation

8. ERD, DFD

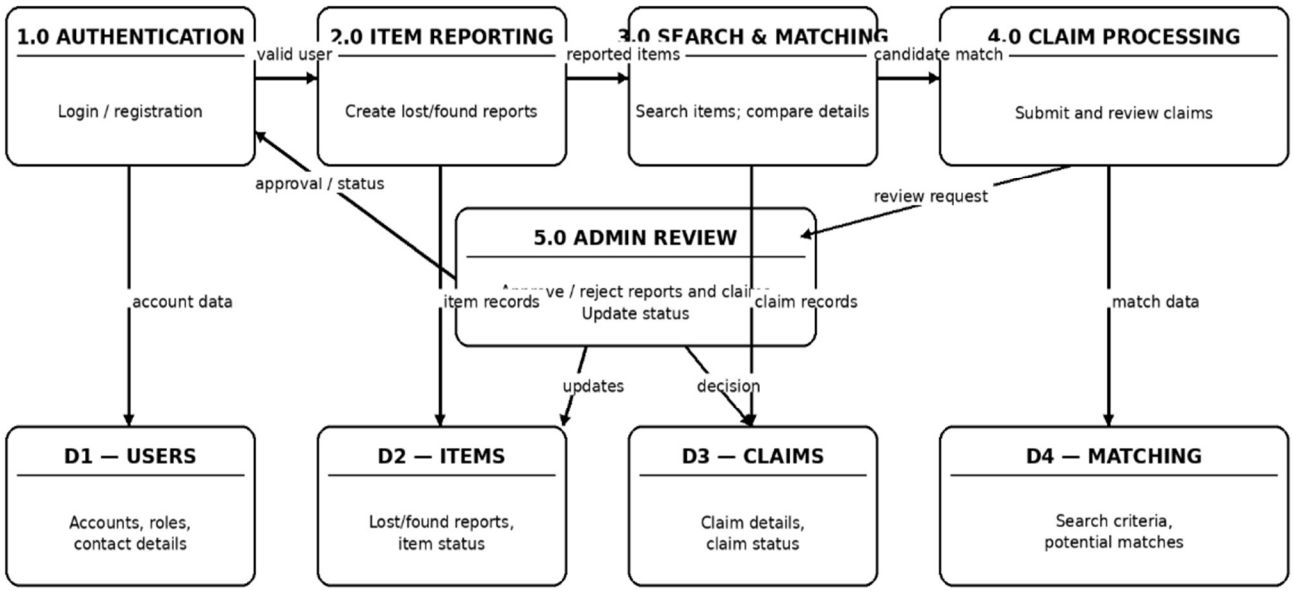
8.1 Entity Relationship Diagram (ERD)



8.2 Data Flow Diagram — Level 0



8.3 Data Flow Diagram — Level 1



9. Process Involved, Algorithm, Flowchart, Database Diagram

9.1 Main Process

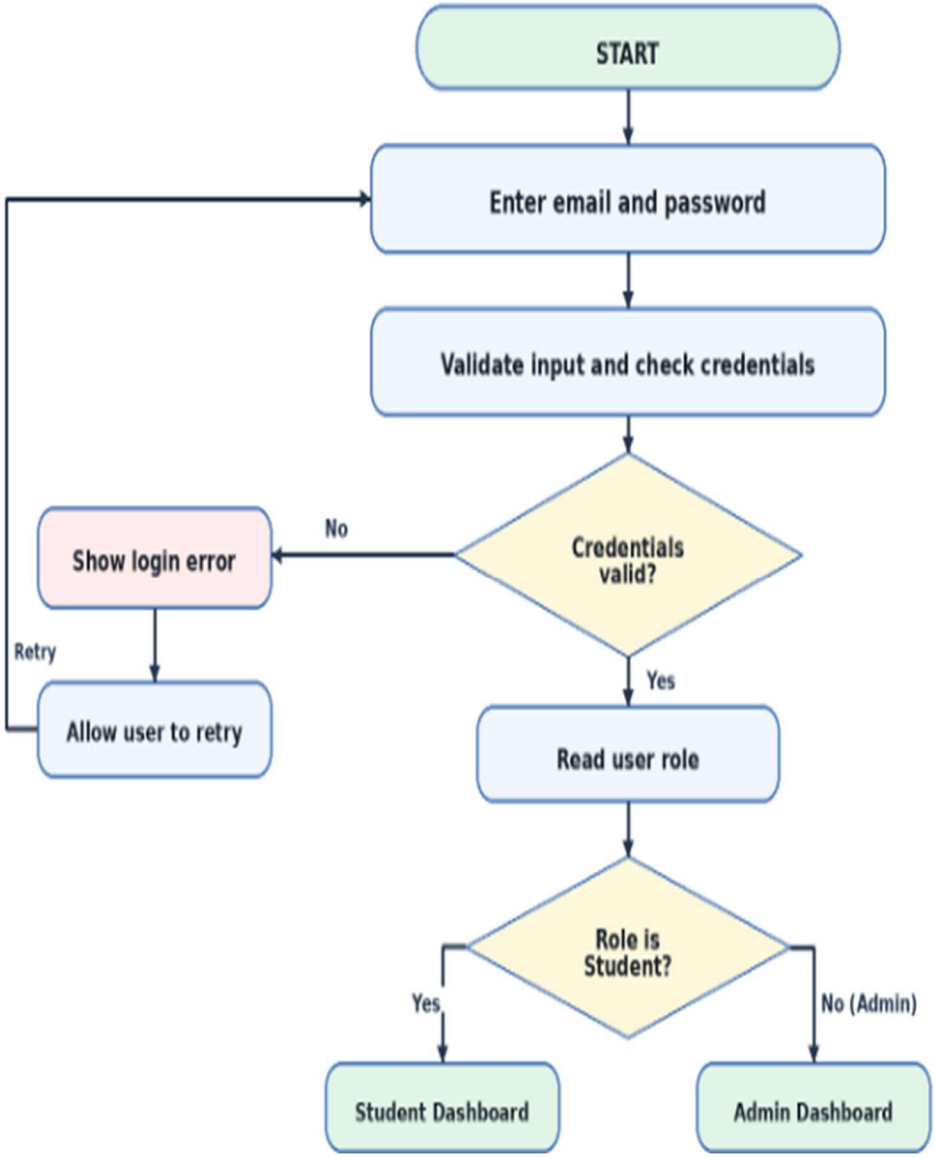
- The user opens the application and logs in or registers as a student.
- The application checks the user's credentials and role.
- The student submits lost/found details or searches existing records.
- The application validates inputs and saves accepted form data through a DAO.
- MatchingService compares relevant lost and found item attributes and displays potential matches.
- If a claim is submitted, its details and current status are stored.
- The administrator reviews relevant reports/claims and updates the status.
- The student checks the updated status and logs out.

9.2 Matching Algorithm

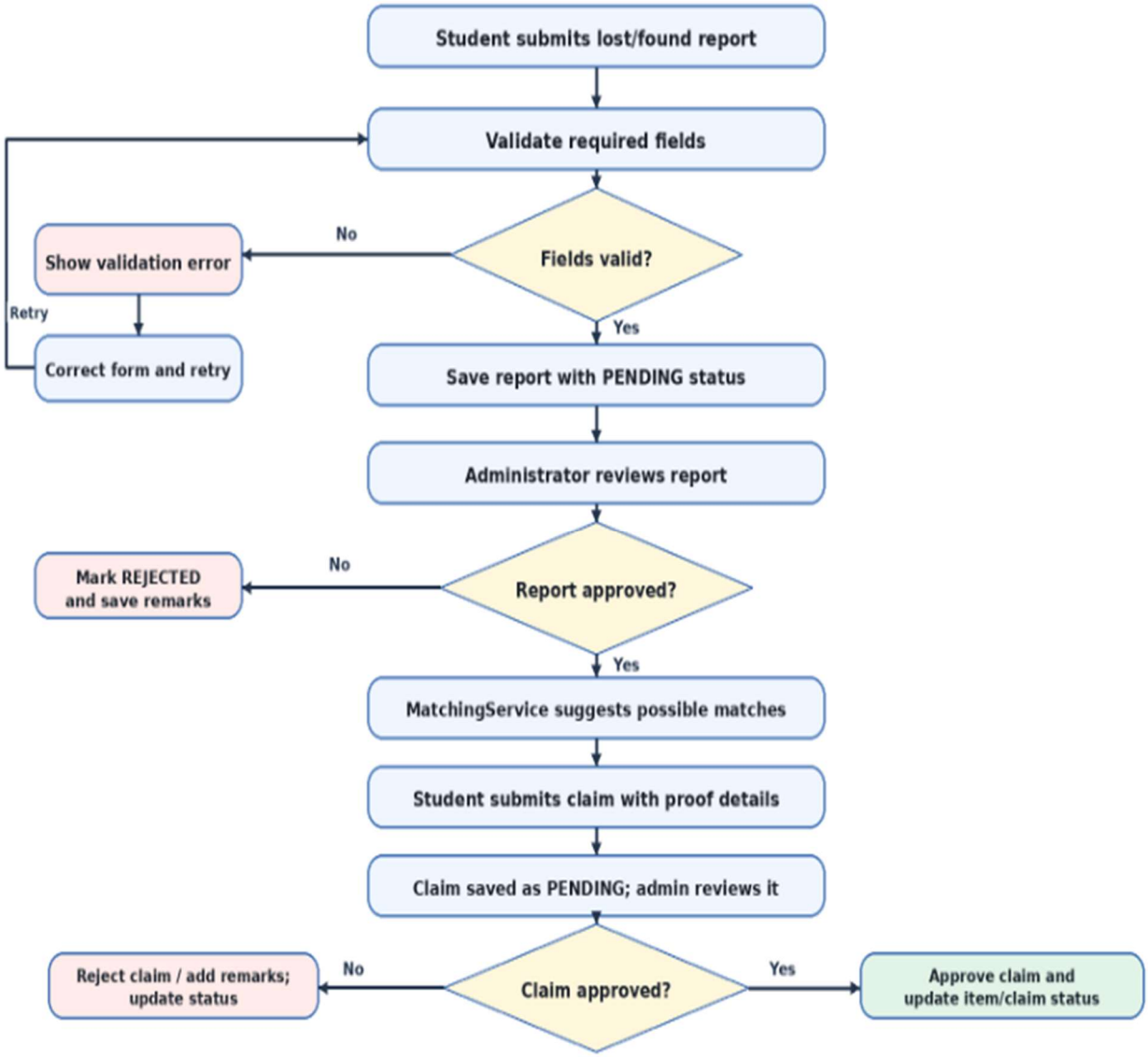
1. Input: one lost-item record and a set of found-item records.
2. Initialize score = 0 for each candidate found item.
3. Compare item names and add up to 30 points based on similarity.
4. Compare categories and add up to 20 points.
5. Compare locations and add up to 25 points.
6. Compare descriptions and add up to 25 points.
7. Keep the total within 0–100 and rank candidates by score.
8. Display high-scoring candidates as possible matches for manual checking.

The exact thresholds for deciding which candidates are displayed should be taken from the final MatchingService.java configuration. A high score must not automatically establish ownership.

9.3 Login Flowchart



9.4 Report and Claim Flow



9.5 Database Tables

Table	Purpose	Key fields
users	Stores registered users and administrator account	id (PK), email (unique), role, college_id
lost_items	Stores lost-item reports	id (PK), user_id (FK), item_name, category, lost_date, status
found_items	Stores found-item reports	id (PK), user_id (FK), item_name, category, found_date, status
claims	Stores claims and review status	id (PK), claimant_id (FK), found_item_id (FK), proof_details, claim_status

10. Input and Output Screen Design (Screenshots)

The following screens are taken from the preview images included in the supplied project archive. They illustrate the intended interface; use screenshots captured from the final running build if the instructor requires evidence of actual execution.

10.1 Login and Student Registration — Input Screens

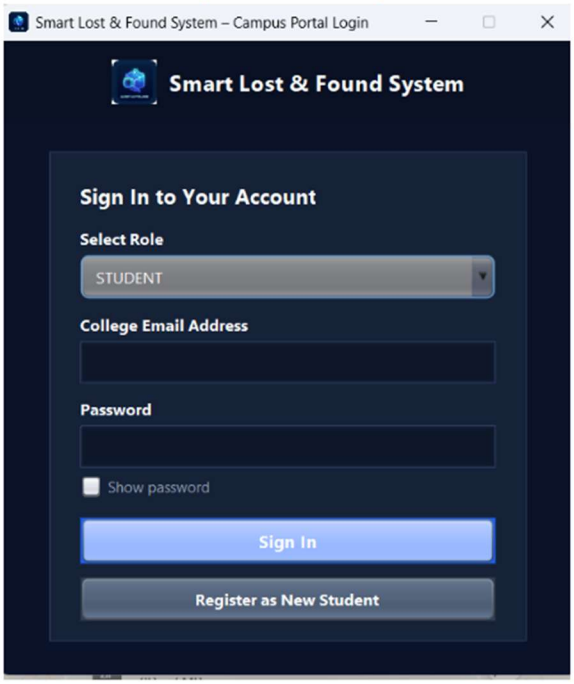


Figure 10.1: Campus Portal Login Screen

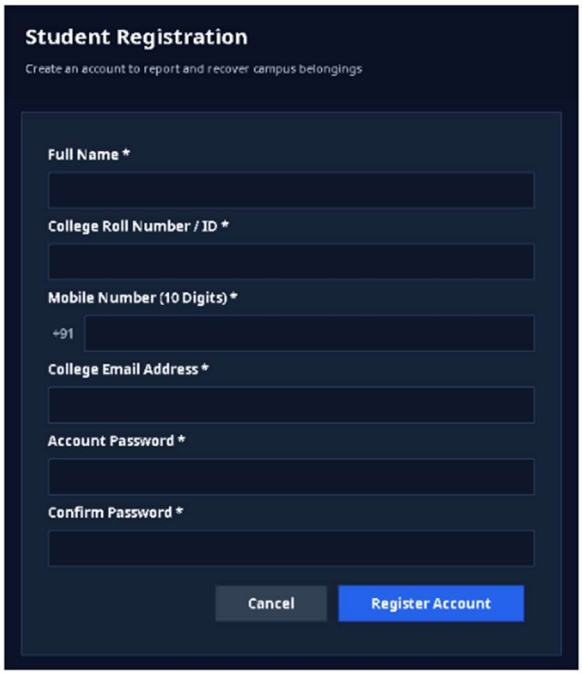


Figure 10.2: Student Registration Screen

The user selects Student or Admin, enters a college email address and password, and chooses Sign In. New students can open registration from the same screen.

The registration dialog collects full name, college roll number/ID, mobile number, college email, password, and password confirmation. Required fields and matching passwords are validated before the account is saved.

10.2 Student Report Lost Item — Input Screen

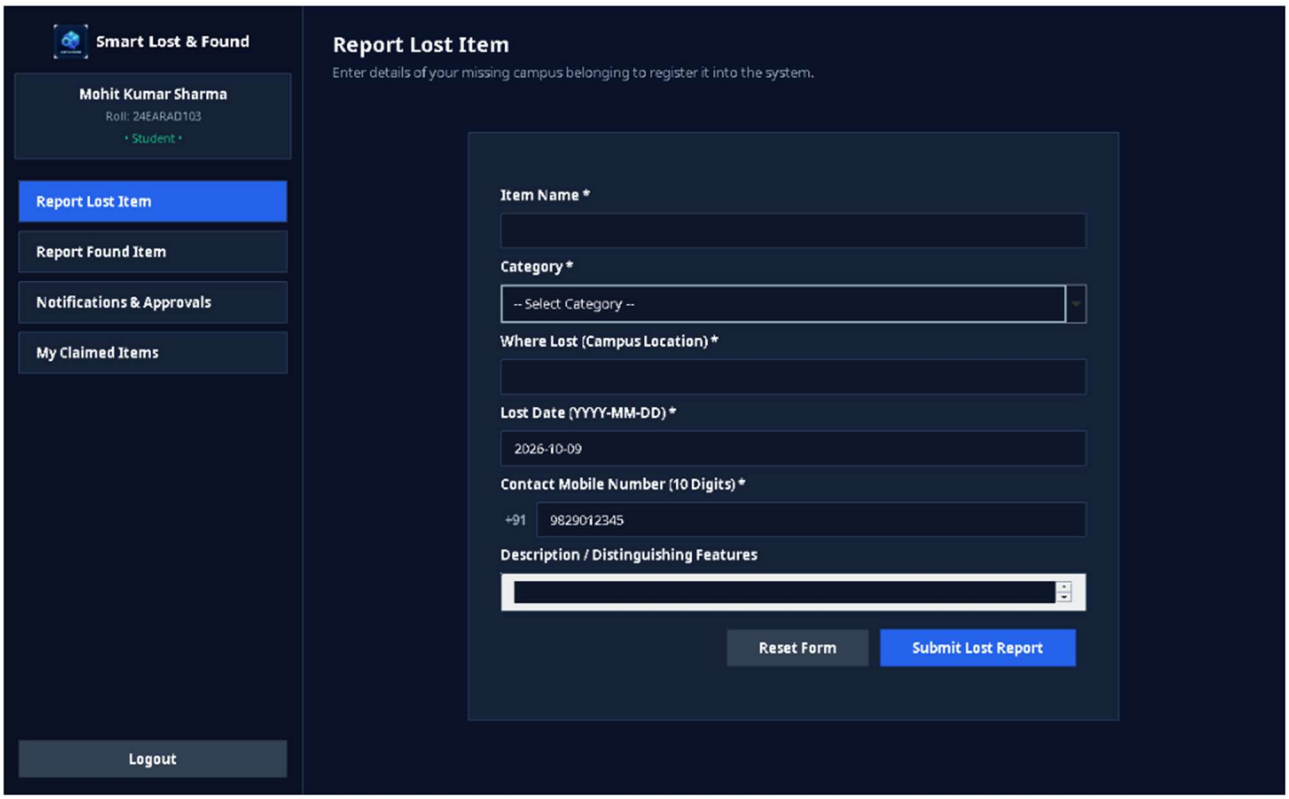


Figure 10.3: Student Report Lost Item

This screen accepts the lost item name, category, campus location, lost date, contact mobile number, and optional distinguishing description. The Reset Form button clears the inputs, while Submit Lost Report sends the report for storage with an initial pending status.

10.3 Student Report Found Item — Input Screen

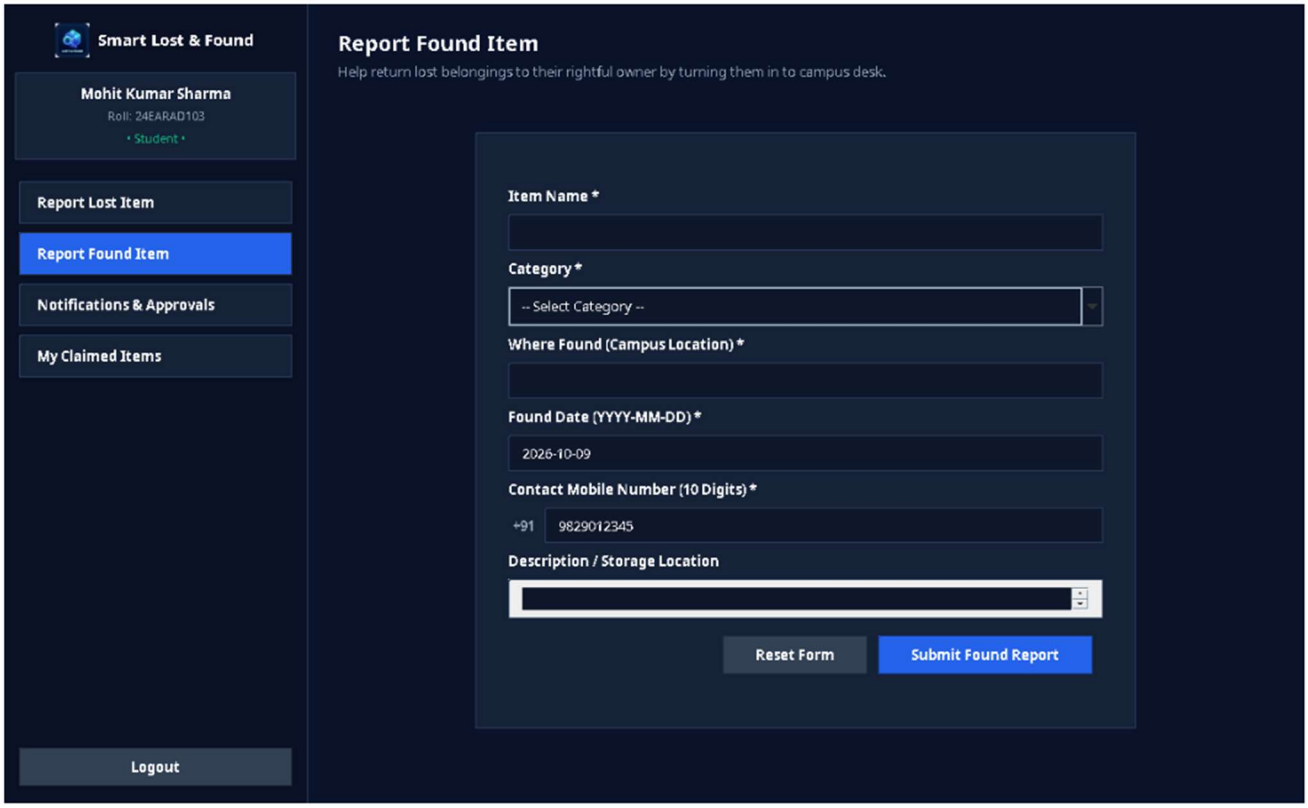


Figure 10.4: Student Report Found Item

This form records an item found on campus. The student enters the item name, category, found location, date, contact number, and description or storage details, then submits the report for administrative review.

10.4 Student Notifications & Approvals — Output Screen

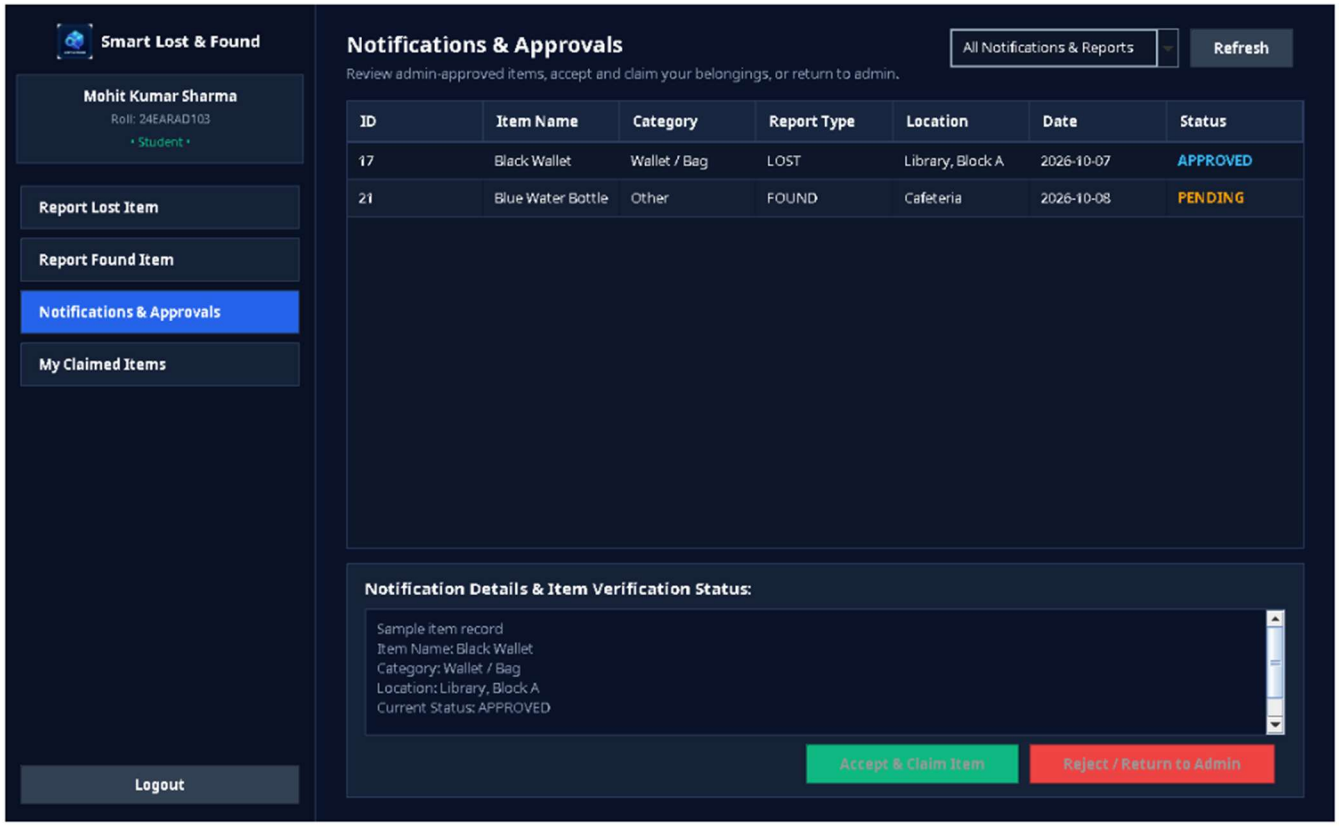


Figure 10.5: Student Notifications & Approvals

This screen lists a student's lost and found reports with report type, item details, location, date, and current status. An approved record can be selected for the available claim action; rejected or pending records show their corresponding status.

10.5 Student My Claimed Items — Output Screen

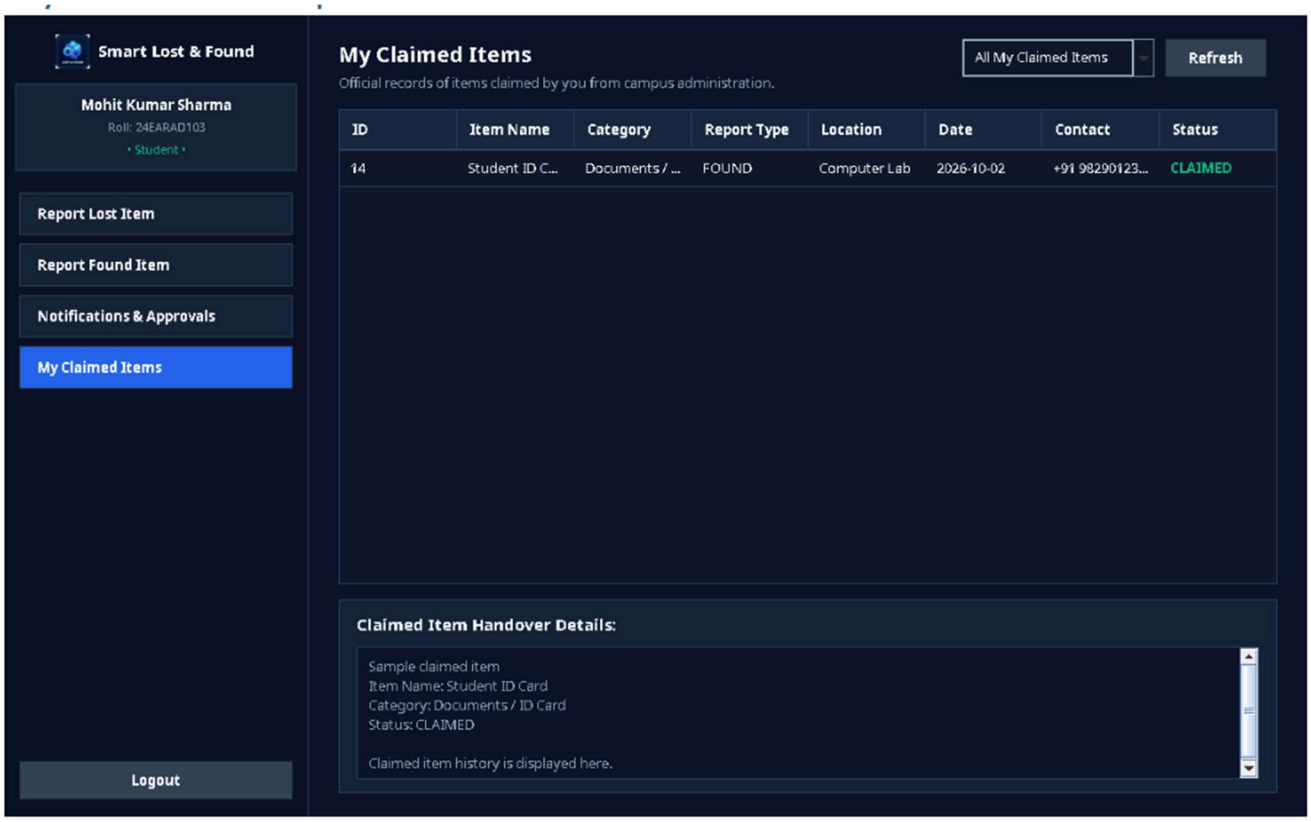


Figure 10.6: Student My Claimed Items

This screen displays items that the student has successfully claimed. The table presents the item name, category, report type, location, date, contact, and claim status so the student can review the claim history.

10.6 Admin Registered Users — Output Screen

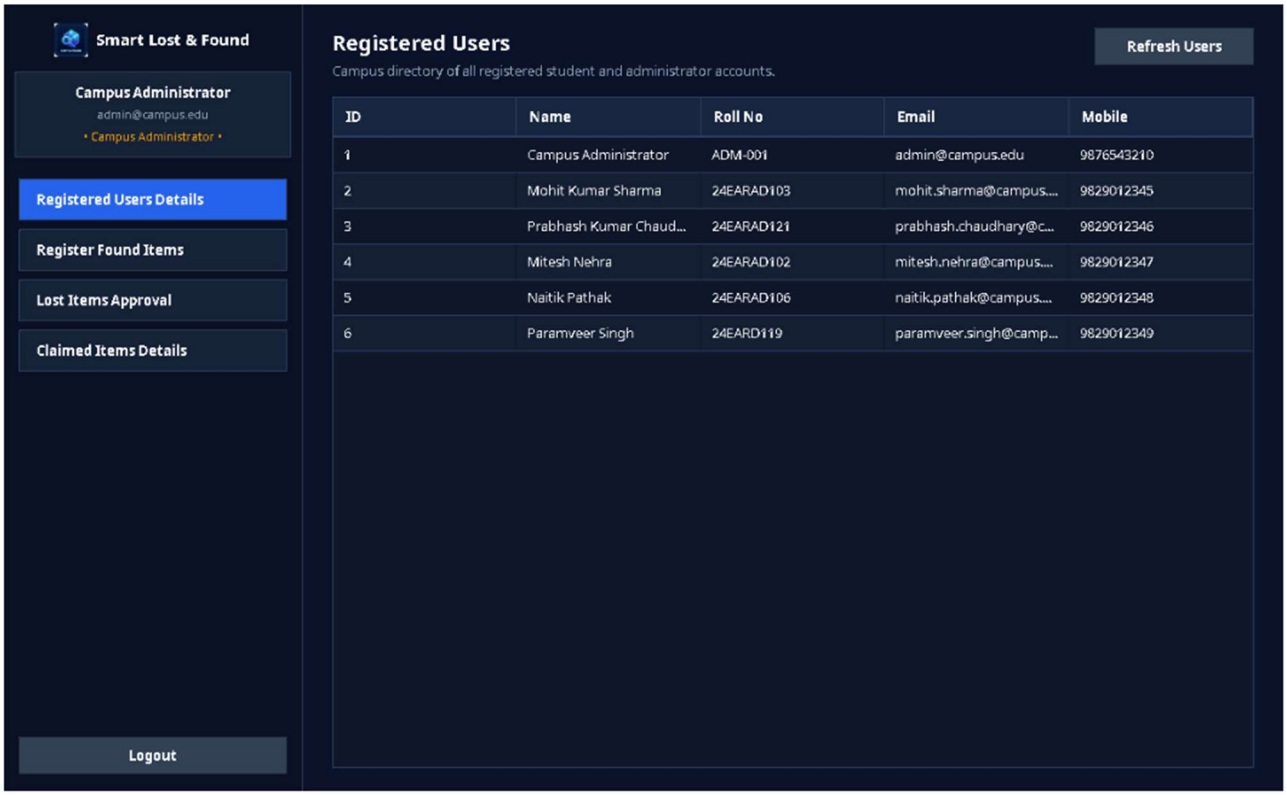
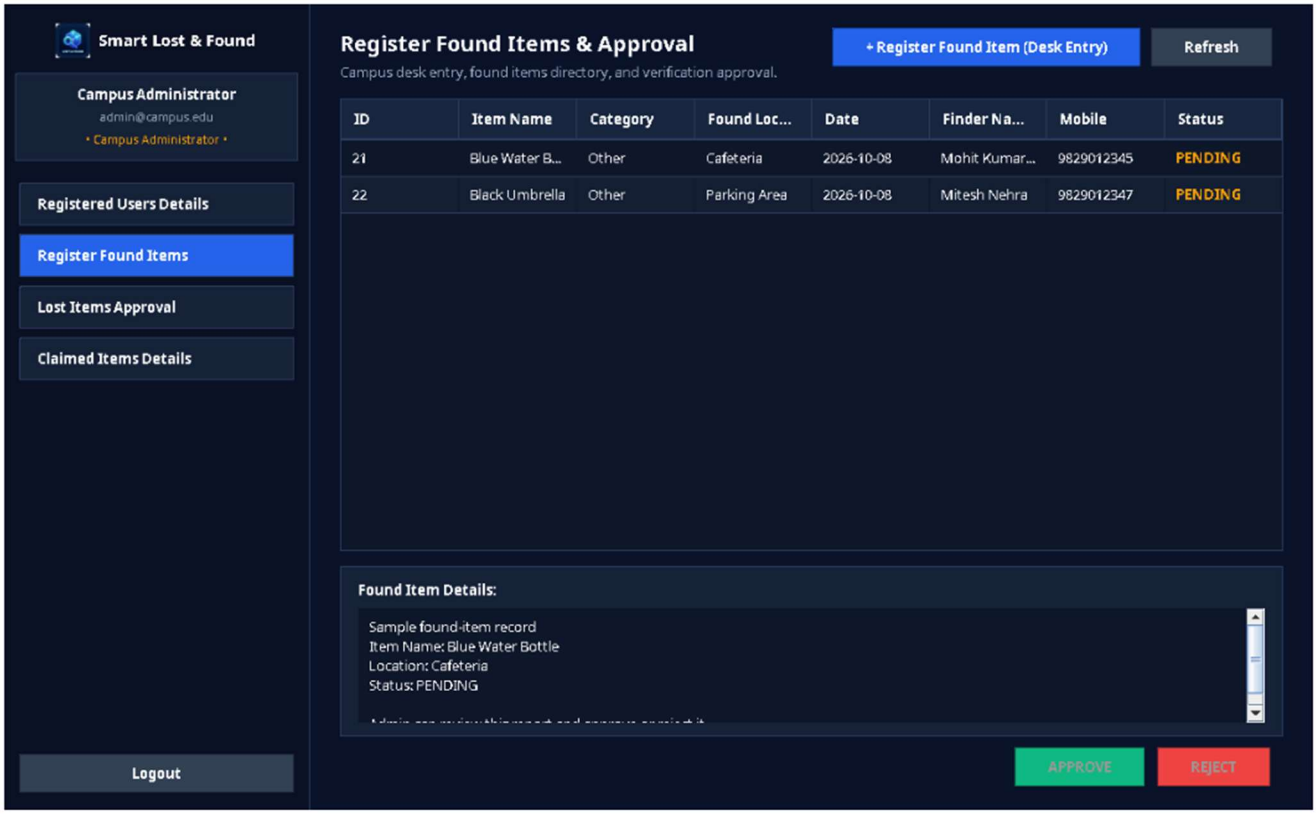


Figure 10.7: Admin Registered Users

The administrator can view the campus user directory. The table displays user ID, name, roll number or college ID, email address, and mobile number. The Refresh Users button reloads the latest user records.

10.7 Admin Register Found Items & Approval — Input/Output Screen



`

Figure 10.8: Admin Register Found Items & Approval

This administrative screen shows the found-item directory and review details. The administrator can open the desk-entry form for a newly received item, refresh the directory, inspect a selected record, and use Approve or Reject to update its status.

10.8 Admin Campus Desk Found Item Entry — Input Screen

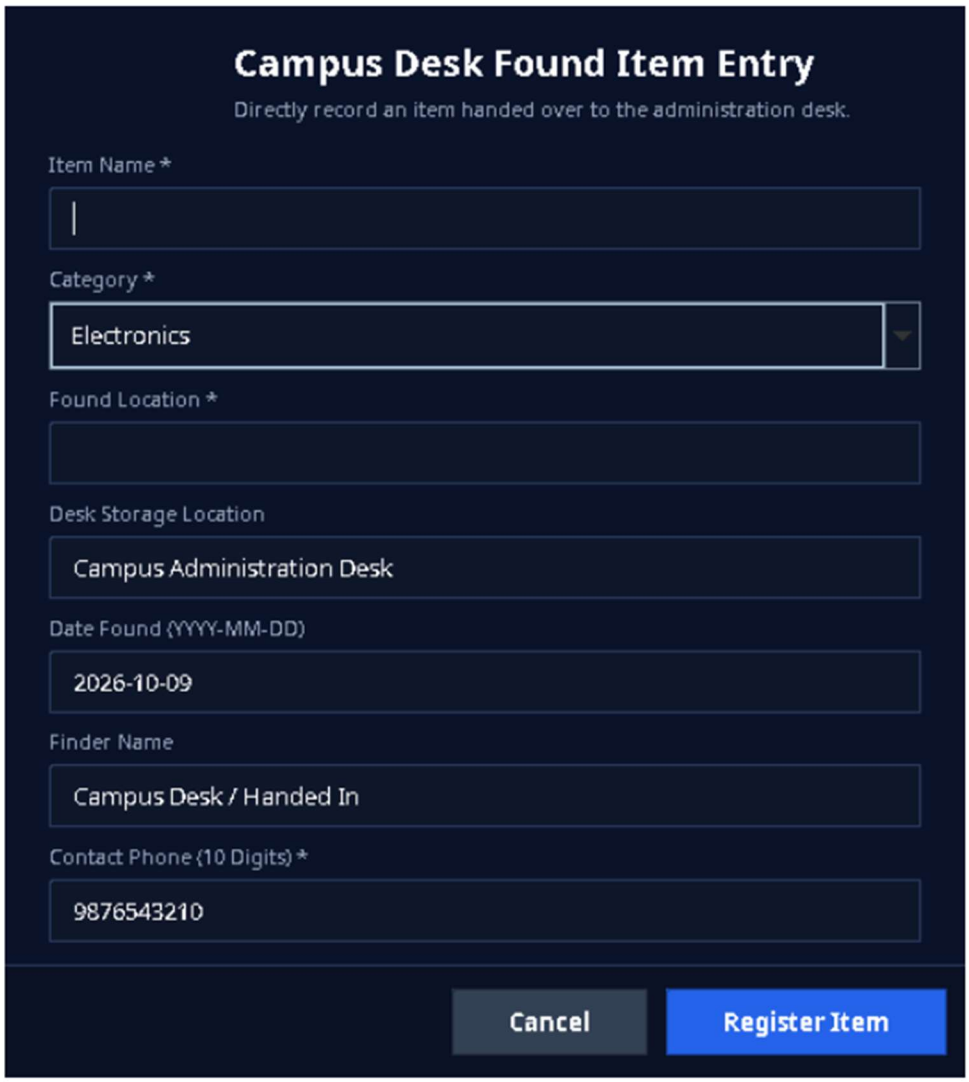


Figure 10.9: Admin Campus Desk Found Item Entry

This dialog lets an administrator enter a found item handed directly to the campus help desk. The visible fields include item name, category, found location, desk storage location, date, finder name, and contact phone. The current source also defines a description field, but it is clipped in this capture at the dialog's configured height and should be checked before final submission.

10.9 Admin Lost Items Approval — Output/Review Screen

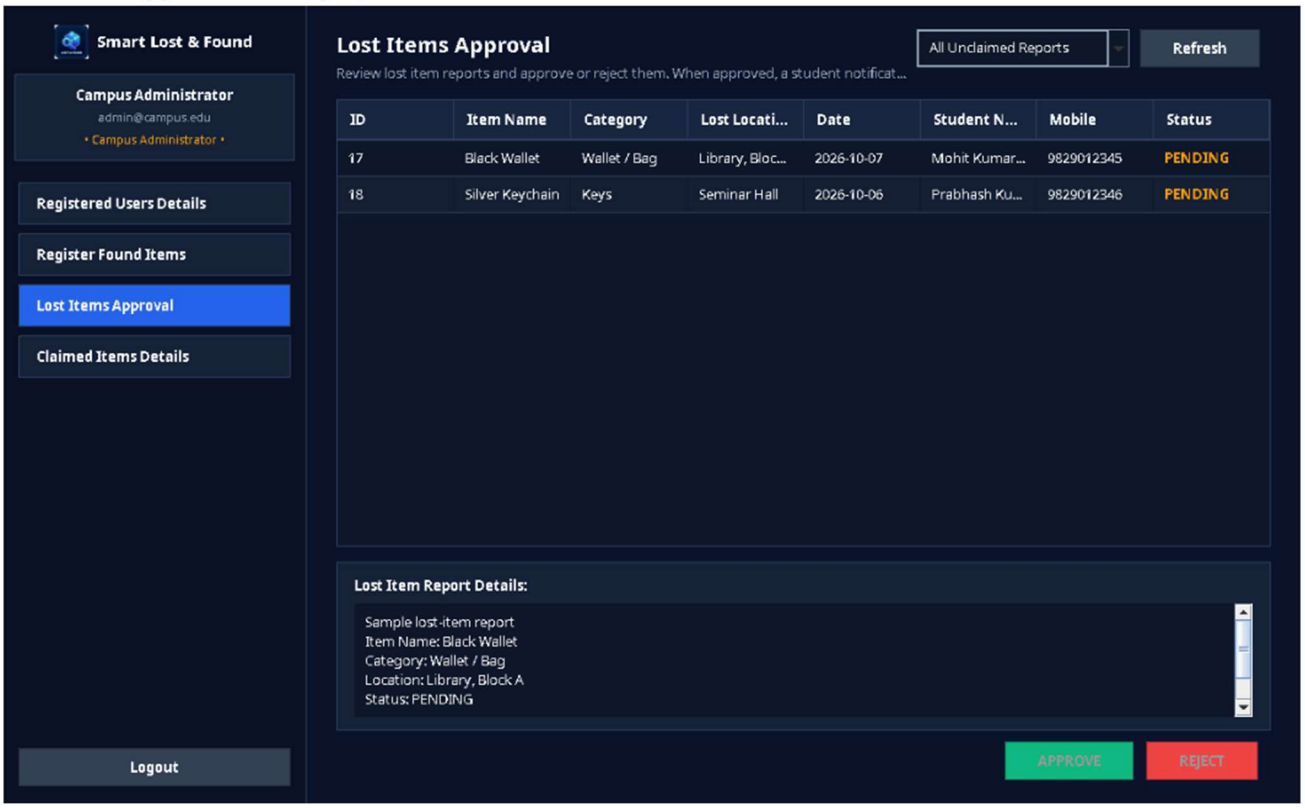


Figure 10.10: Admin Lost Items Approval

This screen lists lost-item reports awaiting administrative action, including item name, category, location, date, student name, contact number, and status. The administrator reviews the details before approving or rejecting a report.

10.10 Admin Claimed Items Details — Output Screen

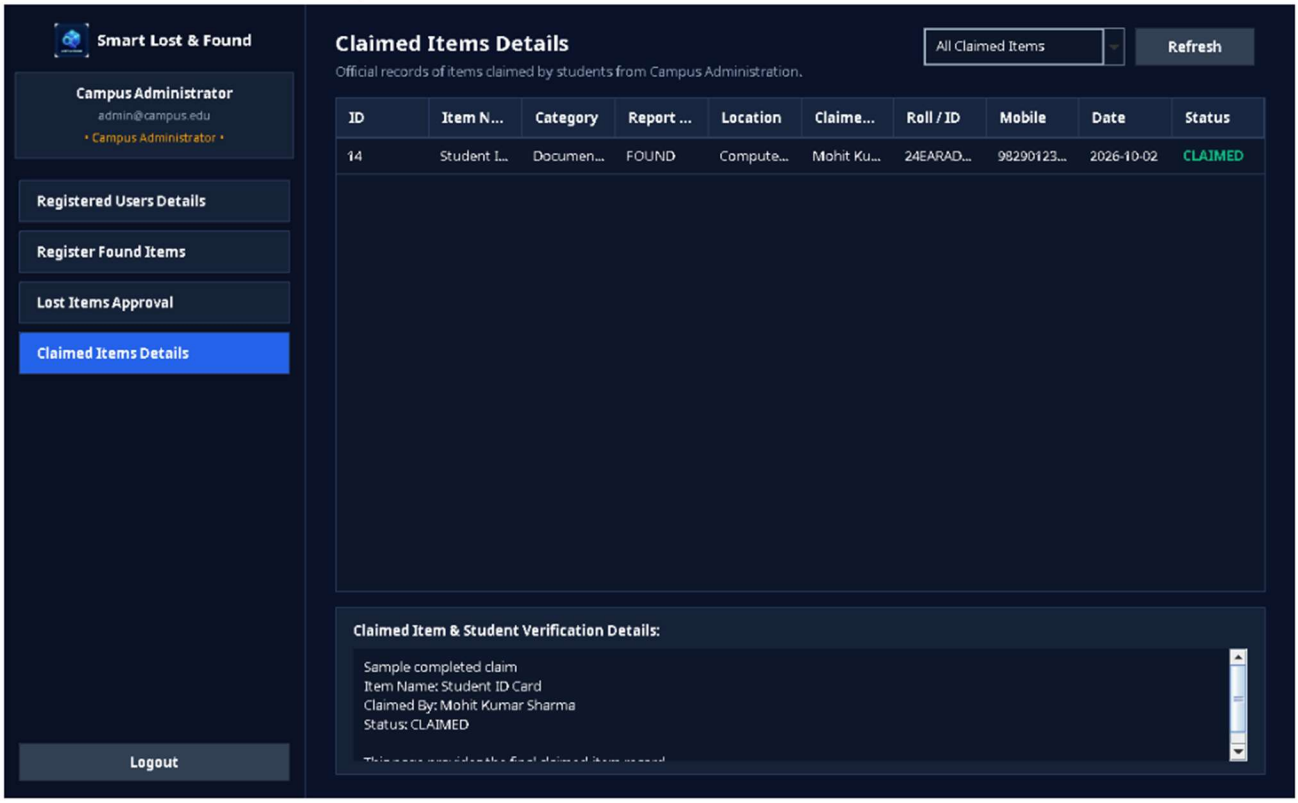


Figure 10.11: Admin Claimed Items Details

This screen provides the administrative record of claimed items, including item details, report type, location, claimant identity, roll number or ID, mobile number, date, and status.

11. Printout of the Code Sheet (Coding)

The complete source code is supplied separately in the project folder. The report includes representative excerpts and module descriptions; the final printed code appendix should include the exact source files from the submitted build.

11.1 Main Components

File / Package	Responsibility
Main.java	Application entry point and initial screen launch
ui/LoginFrame.java	Login interface and authentication interaction
ui/RegisterDialog.java	Student registration interface
ui/DashboardFrame.java	Role-based dashboard and navigation
ui/UITheme.java	Reusable UI styling and component appearance
model/User.java	User data representation
model/LostItem.java	Lost-item data representation
model/FoundItem.java	Found-item data representation
model/Claim.java	Claim data representation
database/DatabaseConnection.java	MySQL connection and database setup support
dao/UserDAO.java	User queries and registration/login persistence
dao/LostItemDAO.java	Lost-item persistence and queries

dao/FoundItemDAO.java	Found-item persistence and queries
dao/ClaimDAO.java	Claim persistence and status operations
service/MatchingService.java	Rule-based possible-match scoring

11.2 Representative JDBC Pattern

```
String sql = "SELECT id, full_name, email, role FROM users WHERE email = ?";
try (Connection con = DatabaseConnection.getConnection();
PreparedStatement ps = con.prepareStatement(sql)) {
ps.setString(1, email);
try (ResultSet rs = ps.executeQuery()) {
if (rs.next()) {
// Map the result to a User model object.
}
}
}
```

This is an illustrative JDBC pattern, not a verbatim extract from a specific source file. For the final code-sheet printout, copy the actual code from the project source so the appendix exactly matches the submitted version.

12. Testing

Testing should be carried out on the final integrated build with the configured MySQL database. The cases below form a practical test checklist. Mark the actual result and pass/fail status after running each case; do not claim a test passed unless it has been executed.

Test ID	Test case	Expected result	Actual result
TC-01	Launch application with database available	Login window opens without error	To be filled
TC-02	Login with valid student credentials	Student dashboard opens	To be filled
TC-03	Login with valid administrator credentials	Admin dashboard opens	To be filled
TC-04	Login with invalid password	Error message; no dashboard access	To be filled
TC-05	Register with valid details	Account is saved and success feedback appears	To be filled
TC-06	Register with duplicate email	Registration is rejected	To be filled
TC-07	Submit lost-item form with required fields	Record is saved and appears in the appropriate list	To be filled

TC-08	Submit form with missing required fields	Validation message appears; record is not saved	To be filled
TC-09	Submit a found-item report	Record is saved and visible for re-view/search	To be filled
TC-10	Search by item key-word/category/loca-tion	Relevant records are displayed	To be filled
TC-11	Run possible-match search	Candidate matches and score are dis-played	To be filled
TC-12	Submit claim with proof details	Claim is stored with pending status	To be filled
TC-13	Admin updates a report/claim status	Database and rele-vant views show updated status	To be filled
TC-14	Close and reopen application	Persisted records remain available	To be filled
TC-15	Enter SQL-like text in a form	Input is treated as data, not executable SQL	To be filled
TC-16	Logout	Session ends and login screen is shown	To be filled

12.1 Types of Testing

- Unit testing: check individual validation, model, DAO and matching methods.
- Integration testing: check that UI actions call the correct service/DAO and database operations.
- System testing: run complete student and administrator workflows.
- Validation testing: check empty fields, invalid dates, malformed email/phone and duplicate registration.
- Regression testing: rerun important cases after a code change.

13. User / Operational Manual

13.1 Prerequisites

- Install a compatible JDK (the project README specifies JDK 17 for rebuilding).
- Install and start MySQL Server.
- Create/configure the smart_lost_found database using the supplied SQL script or the application's database initialization flow.
- Check db.properties and confirm host, port, database name, username, and password.
- Keep MySQL Connector/J available on the application classpath.

13.2 Starting the Application

- Open the project folder and verify the database settings.
- Run the provided run.bat launcher, or compile/package using build.bat if source changes are made.
- Wait for the Login screen to appear.
- If a database connection error occurs, verify that MySQL is running and credentials are correct.

13.3 Student Operations

- Register with the required details if an account has not already been created.
- Log in using the registered email and password.
- Open the Report Lost or Report Found form and complete all required fields.
- Submit the form and check the confirmation message.
- Use search/matching features to inspect relevant records and possible matches.
- Submit a claim with truthful proof details when appropriate.
- Check the report or claim status from the dashboard and log out when finished.

13.4 Administrator Operations

- Log in using the authorized administrator account.
- Open the appropriate user, lost-item, found-item, or claim section.
- Review the item details and any claim proof/remarks available.
- Approve or reject the relevant record using the provided action controls.
- Confirm the status change and re-check the table after refresh.
- Log out after completing administrative work.

13.5 Security, Access Rights, Backup and Controls

- Use a separate administrator account and do not share its credentials.
- Restrict administrative actions to the Admin role.
- Use PreparedStatement for SQL statements containing user input.
- Do not store database passwords in public repositories; protect db.properties on shared machines.
- Back up the database regularly using a MySQL export/dump and verify that the backup can be restored.
- Avoid inserting fabricated test records into the final demonstration database; use clearly labelled test data only in a separate test database.

- Log out when leaving a shared computer and do not expose personal contact details unnecessarily.
- Important: the supplied SQL file currently includes a fixed administrator and five fixed student records. If the final project requirement is that only the administrator is predefined and students are created through registration, remove the student seed rows and verify the final initialization script before submission.

14. Conclusions

The Smart Lost & Found System provides an efficient digital solution for managing lost and found belongings within a college campus. The main objective of this project is to simplify the process of reporting lost items, registering found items, searching for records, identifying possible matches, and managing claims through a centralized application.

The application is developed using Java Swing to provide a user-friendly graphical interface with separate functionalities for students and administrators. Students can register, log in, submit lost and found item reports, search for available records, view possible matches, submit claims with supporting details, and track their claim status. Administrators can review submitted reports and claims, approve or reject requests, and update their statuses.

The system uses MySQL for data storage and JDBC to establish communication between the Java application and the database. The DAO (Data Access Object) design pattern separates database operations from the user interface, making the application more organized, maintainable, and easier to modify. Additionally, the rule-based MatchingService compares item details, such as names, categories, locations, and descriptions, to suggest possible matches. These suggestions help users identify relevant records, while final ownership verification requires appropriate evidence.

This project demonstrates the practical application of object-oriented programming, GUI development, database management, SQL, JDBC connectivity, modular programming, and software testing. It provides a structured alternative to traditional manual methods of managing lost and found items in educational institutions.

In conclusion, the Smart Lost & Found System offers a useful foundation for improving the management of lost and found belongings on campus. With future enhancements such as improved security, automated notifications, image support, and more accurate matching, the system can become more reliable and convenient for students and administrators.

15. Future Enhancement

- Add password hashing and a secure password-reset workflow.
- Improve match ranking with configurable thresholds and more robust text similarity.
- Add item photographs with controlled image storage and privacy safeguards.
- Provide email or in-app notifications when a possible match or claim status changes.
- Add audit logs for administrative decisions and status changes.
- Add exportable reports and dashboard statistics for campus administrators.
- Support backup/restore from the admin interface.
- Improve accessibility, keyboard navigation, and responsive layout behavior.
- Add automated tests for DAO and service classes.
- Deploy a centralized version for multiple campus computers while protecting database access.

16. References

- Oracle. Java Platform, Standard Edition Documentation. https://docs.oracle.com/en/java/
- Oracle. Java Swing Tutorial. https://docs.oracle.com/javase/tutorial/uiswing/
- Oracle. JDBC Basics. https://docs.oracle.com/javase/tutorial/jdbc/basics/
- MySQL. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/
- MySQL. MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/
- Project source files: SmartLostAndFound/README.md, database/database.sql, and Java source files under SmartLostAndFound/src/.